

Chapter 17: Under the Hood

"Any sufficiently advanced technology is indistinguishable from magic."
— Arthur C. Clarke

"Any sufficiently analyzed magic is indistinguishable from technology."
— The corollary that matters for this chapter

You have spent sixteen chapters learning to *use* Microsoft 365 Copilot. This chapter is different. This chapter is about understanding *how it works* — not because you need to become an engineer, but because understanding the machinery makes you dramatically better at operating it.

Here is why this matters at GES specifically: when you understand that Copilot's answers come from a structured web of relationships rather than a generic knowledge base, you start asking different questions. When you understand that a tool call is a structured request the model emits rather than a command it executes, the permission model suddenly makes sense. When you understand that retrieval can fail silently, you stop blaming the AI for answers it was never given the ingredients to produce.

Chapter 1 introduced the flashlight theory — that context is what turns raw intelligence into useful output. This chapter shows you what the flashlight is actually made of. It shows you the data structure underneath, the retrieval system that finds the right content, and the tool architecture that lets Copilot act rather than just talk.

Three big ideas organize everything that follows:

1. **The Graph.** Your organization is not a pile of files. It is a web of relationships between people, documents, meetings, messages, and sites. Microsoft Graph is the unified layer that holds those relationships, and the semantic index is what makes them searchable by meaning rather than keyword.

2. **Tools are the unlock.** A language model alone can only produce text. A model with tools can act: query a database, send a message, book a room, run a workflow. Function calling — the ability to emit a structured request for action — is the single biggest capability jump in the AI era.
3. **Adding your own tools.** Graph connectors bring GES line-of-business data into Copilot's reach. Copilot Studio lets you build custom agents with custom actions. The Model Context Protocol (MCP) is the emerging open standard. Power Automate flows become callable tools. This is where Copilot stops being Microsoft's product and becomes yours.

By the end of this chapter, you will understand why Copilot can answer questions about your organization that a generic chatbot never could — and why it sometimes cannot answer questions that seem obvious. That understanding is practical, not academic. It changes how you prompt, how you troubleshoot, and how you decide what to build.

1. The Graph: Your Organization as a Web of Relationships

1.1 What Microsoft Graph actually is

Strip away the marketing and here is the core truth: **Microsoft Graph is a unified API and data layer that connects everything in Microsoft 365.**

Not a database in the traditional sense. Not a folder hierarchy. A *graph* — which in this context means a structure of **nodes** (things) connected by **edges** (relationships between things).

The nodes are the entities you work with every day:

- **People** — every user in your organization
- **Documents** — files in SharePoint, OneDrive, Teams
- **Messages** — emails in Exchange, chats in Teams
- **Meetings** — calendar events with attendees and recordings
- **Sites** — SharePoint sites and Teams channels
- **Groups** — Microsoft 365 groups and distribution lists

The edges are the relationships between them:

- Who **authored** a document
 - Who **attended** a meeting
 - Who **shared** a file with whom
 - Who **replied to** an email thread
 - Who **works with** whom (inferred from collaboration patterns)
 - Which document **belongs to** which site
 - Which meeting **relates to** which project
-

Here is what makes this powerful: **the relationships are first-class data, not afterthoughts.**

In a traditional file system, a document is just a document. It sits in a folder. The fact that you wrote it, that your colleague edited it, that it was attached to an email chain about a specific show, that it was discussed in a Tuesday morning meeting — all of that context exists only in human memory or in timestamps you'd have to reconstruct manually.

In Microsoft Graph, all of that context is *explicit and queryable*. The document node is connected to a person node (author) by an "authored" edge. It's connected to other person nodes by "shared with" edges. It's connected to an email thread node. It's connected to a meeting node where it was a shared resource.

When Copilot answers a question about your work, it is not searching a pile of files. It is traversing a web of relationships — following the edges from what you asked to what is relevant, respecting your permissions at every step.

1.2 The show floor as a graph — an analogy you already know

If this sounds abstract, consider something you understand deeply: a show floor.

A show floor is not a list of booths. It is a *graph* of relationships.

The **nodes** are:

- Exhibitors (companies)

- Booths (physical spaces)
- Aisles (pathways)
- Freight (crates, pallets, equipment)
- Labor (crews, unions)
- Venues (the building itself)
- Services (power, rigging, graphics)

The **edges** are:

- This exhibitor **occupies** this booth
- This booth **faces** this aisle
- This freight **belongs to** this exhibitor
- This freight **arrived at** this dock
- This labor crew **is assigned to** this booth
- This booth **requires** this service
- This aisle **connects to** these booths
- This venue **has** these dock constraints

When a show manager walks a floor during move-in, they are traversing this graph instinctively. "The carpet crew is in Aisle 400 → those booths connect to docks 3 and 4 → the freight for those exhibitors arrived this morning → the labor call for that section starts at noon." That is graph traversal — following edges from node to node to understand how everything connects.

Microsoft Graph does the same thing with your organizational data. When you ask Copilot "What did we decide about the move-in sequence for the Meridian show?", it traverses: your recent meetings → attendees → shared documents → email threads — following the edges to find content that is *relevant to your question*, not just keyword-matched to your words.

1.3 Why the Graph is the entire reason Copilot beats a generic chatbot

Here is the insight that changes how you think about Copilot:

A generic AI chatbot knows language. Copilot knows your organization.

ChatGPT, Claude, Gemini — they are trained on the public internet. They know what an exhibitor service kit *is* in the abstract. They do not know what *your* exhibitor service kit says, who wrote it, which show it belongs to, or that it was updated last Tuesday after a conversation with the organizer.

Copilot does. Because Copilot is grounded in Microsoft Graph — your Graph, your tenant, your relationships.

When you ask Copilot to summarize the status of a show, it can pull from:

- The email threads between you and the organizer
- The Teams channel where the show team discussed move-in
- The SharePoint site where the operations manual lives
- The meeting where the freight targets were finalized
- The document where the labor forecast was revised

It does not just know that these things exist. It knows *how they relate* — that this email is a reply to that earlier thread, that this document was shared in that meeting, that this person is the account director and that person is the operations lead.

That relational knowledge is what makes Copilot's answers specific rather than generic. It is also what makes the permission model work — the Graph knows not just what exists, but who is allowed to see it.

2. The Semantic Index: Making the Graph Searchable by Meaning

The Graph holds the relationships. The **semantic index** makes them findable.

2.1 From keywords to meaning

Traditional search works by keyword matching. You type "freight target" and the system finds documents containing those exact words. The problem is obvious: a document might discuss delivery schedules, receiving windows, and material handling timing without ever using the phrase "freight target" — and traditional search would miss it.

The semantic index solves this with **vector embeddings** — mathematical representations of meaning rather than text.

Here is the intuition: imagine every document, every email, every meeting transcript is converted into a point in a high-dimensional space. Documents that mean similar things end up near each other, even if they use different words. "Freight target," "delivery window," "receiving deadline," and "advance warehouse cutoff" all cluster together because they are about the same *concept*.

When you ask Copilot a question, your question is also converted into a vector. The system then finds the Graph content whose vectors are *closest* to your question — the content that is semantically most relevant, regardless of exact word choice.

This is why Copilot can answer "What's the plan for getting freight to the floor on Tuesday?" even if the operations manual uses the phrase "material handling sequence" instead of "freight plan." The semantic index understands that these are the same concept.

2.2 How retrieval actually works — RAG and grounding

When you prompt Copilot, you are not talking directly to the language model. Your question goes through a retrieval pipeline first.

RAG — Retrieval-Augmented Generation — is the technical term for this pattern:

1. **Query understanding.** Copilot interprets your question and determines what kind of information would answer it.
2. **Retrieval from the Graph.** The system queries the semantic index, following relationships in the Graph to find content that is relevant to your question *and* that you have permission to see.
3. **Grounding.** The retrieved content is assembled into context — the "flashlight beam" from Chapter 1. This context accompanies your question when it goes to the language model.

4. **Generation.** The language model produces an answer, grounded in the retrieved content. When Copilot cites a source, it is citing what the retrieval system found.

This architecture is why Copilot's answers are specific to your organization. The language model provides the reasoning; the Graph provides the facts. Without the Graph, you would get generic answers. Without the language model, you would get search results without synthesis.

2.3 Permission trimming — the Graph enforces access at retrieval time

Here is a fact that should give you confidence: **the Graph enforces your permissions at retrieval time, before any content reaches the language model.**

Chapter 12 covered this from the SharePoint governance perspective. Here is the technical mechanism underneath:

When retrieval queries the Graph on your behalf, it runs *as you* — with your identity, your permissions, your access rights. Content you cannot see in SharePoint directly, you cannot see through Copilot. The filtering happens at the data layer, not at the presentation layer.

This is not a policy overlaid on top of the system. It is the architecture. Microsoft Graph was built with identity and access control as foundational principles because it was designed to serve all of Microsoft 365 — including applications where permission enforcement is legally mandated.

The practical implication: **Copilot cannot show you a document you were never given access to.** If you ask about a client account you are not assigned to, and the answer would require content from that account's SharePoint site, Copilot will not retrieve it. Not because it chose to hide it, but because the retrieval system — running as you — cannot see it.

This is also why oversharing in SharePoint becomes visible through Copilot. If you *do* have access to content you should not (because someone shared too broadly), Copilot will find it and use it. The system enforces the permissions that exist, not the permissions that should exist. That is why Chapter 12's governance guidance matters operationally.

Trust — *we trust each other to always be honest and do what's right.*

The permission model is trustworthy because it is architectural, not behavioral. Copilot does not decide to respect your access rights — it cannot do otherwise. But that trust depends on correct permissions being set in the first place. If a client's confidential material is readable by someone who should not see it, the Graph will serve it faithfully. Trust in the system requires trustworthy inputs.

2.4 Graph connectors — bringing GES data into the Graph

The Graph contains Microsoft 365 data by default. But what about the systems that are not Microsoft 365?

GES runs line-of-business systems for exhibitor orders, labor scheduling, freight tracking, venue specifications, and show management that predate the cloud. That data lives outside the Graph — which means, by default, Copilot cannot see it.

Graph connectors solve this by ingesting external data into the semantic index, making it searchable and retrievable alongside Microsoft 365 content.

A Graph connector does three things:

1. **Ingestion.** It reads data from the external system — a database, a SaaS application, a custom platform.
2. **Mapping.** It converts that data into Graph-compatible entities with properties and relationships.
3. **Indexing.** It feeds the data into the semantic index so it becomes part of what Copilot can retrieve.

The result: Copilot can answer questions that require your line-of-business data, not just your email and documents.

For GES, the connector opportunity is substantial:

- **Venue knowledge base.** Dock configurations, rigging limits, union jurisdictions, freight quirks — indexed and retrievable when a show

manager asks "What do I need to know about this venue?"

- **Labor scheduling system.** Crew assignments, rate structures, availability — so Copilot can ground answers about labor planning in actual scheduling data.
- **Exhibitor order system.** Service orders, booth assignments, add-ons — so exhibitor services can ask "What did this exhibitor order?" and get a real answer.
- **Freight tracking.** Shipment status, advance warehouse receiving, crate manifests — so operations can ask "Where is this exhibitor's freight?" without switching systems.

Microsoft publishes pre-built connectors for common systems (Salesforce, ServiceNow, Jira, and over 1,400 others) and provides a connector SDK for building custom connections to proprietary systems.

Every Graph connector widens the retrieval surface. That is the point — and also the risk.

When you connect an external system to the Graph, its data becomes subject to the same permission model as everything else. If the external system has its own access controls, those controls need to be mapped correctly, or data that was restricted in the source system becomes broadly retrievable through Copilot.

Connectors for GES line-of-business data should be deployed with the same care as SharePoint site permissions. The question is always: "Who should be able to ask Copilot about this, and will the connector enforce that correctly?"

2.5 Honest limits — what the Graph cannot do

Understanding how retrieval works also means understanding when it fails.

If it is not in the Graph, Copilot cannot see it. A document on a local hard drive, a conversation in a non-Microsoft chat system, institutional knowledge that exists only in someone's head — none of this is retrievable. Copilot is grounded in what the Graph contains, and the Graph contains only what has been ingested into it.

Retrieval fails silently. When Copilot cannot find relevant content, it does not say "I searched the Graph and found nothing." It generates an answer from what it *did* find — which may be nothing, in which case you get a generic response, or may be the wrong thing, in which case you get a confidently incorrect response. The absence of a "no results" signal is one of the hardest things to calibrate for.

Stale documents poison answers. The semantic index reflects what is in the Graph today. If the current rate sheet sits next to the superseded one from two years ago, both are indexed, and retrieval may find either. This is the grounding drift problem from Chapter 12 — and it is a retrieval problem at heart.

Recency is a weak signal. The semantic index prioritizes *meaning* over *time*. An older document that is semantically closer to your question may rank above a newer one that is less similar. This is usually correct, but it means "give me the latest" requires explicit phrasing, and even then retrieval may not perfectly capture your intent.

The Graph knows relationships, not truth. A document that is connected to the right people and shared in the right channel will be retrieved — even if its content is wrong. The Graph tells Copilot that this document is relevant; it does not tell Copilot that this document is accurate.

3. Tools Are the Unlock

Everything so far has been about retrieval — finding information and grounding answers in it. Now we turn to the capability that changed everything else: **tools**.

3.1 The fundamental shift — from talking to acting

A language model, by itself, can only produce text. It can reason, summarize, draft, explain — all valuable. But it cannot *do* anything. It cannot send an email, book a room, query a database, run a calculation, or update a record.

Function calling — also called tool use — changes this. It gives a language model the ability to request actions, not just generate text.

Here is the crucial insight: **the model does not execute the action itself**. It emits a *structured request* that a trusted layer executes on its behalf.

When you ask Copilot to send an email, here is what actually happens:

1. **Reasoning.** The model determines that your request requires sending an email and identifies the recipient, subject, and body from your prompt.
2. **Tool call.** The model emits a structured request — essentially a JSON object specifying "call the send_email function with these parameters."
3. **Execution.** The Copilot system receives this request, validates it, and executes the email send using your identity and your Outlook permissions.
4. **Return.** The result (success, failure, confirmation) returns to the model.
5. **Response.** The model incorporates the result into its response to you.

The model decided *what* to do and *how* to specify it. The system did the actual doing. This separation is not a technical detail — it is the entire reason tool use is safe enough to deploy in an enterprise.

3.2 Why this is the single biggest capability unlock

The intelligence has been impressive for years. What changed is that the intelligence can now *work*.

Consider the difference:

Without tools: "Here is a draft of the email you should send to the organizer about the revised move-in sequence. You can copy it and send it yourself."

With tools: "I have sent the email to the organizer about the revised move-in sequence. Here is what I sent."

The second version saves thirty seconds. Multiply by a thousand tasks a day across 2,600 employees, and you are looking at structural time recovery. But the real transformation is not time — it is *what becomes possible*.

Cowork is a tool-use story. When Cowork builds a reconciliation package by pulling data from SharePoint, analyzing it in Excel, drafting a summary in Word, and preparing an email to the organizer, it is executing a sequence of

tool calls — each one a structured request to a specific capability.

Power Automate integration is a tool-use story. When a Copilot agent triggers a flow that updates a record in a line-of-business system, it is calling a tool — the flow itself — with parameters derived from the conversation.

Graph queries are tool calls. When Copilot searches your email or retrieves a document, it is calling the Microsoft Graph API — another tool in the toolkit.

3.3 Tool permissions inherit from the user

Here is the security principle that makes tool use deployable at scale: **every tool executes with your identity and your permissions.**

When Copilot sends an email, it sends from your account, subject to your Outlook configuration. When it retrieves a SharePoint document, it retrieves only what you can access. When it triggers a Power Automate flow, the flow runs with your connection credentials.

The model does not have its own identity. It does not have elevated privileges. It acts *as you*, with *your* rights, subject to *your* limits.

This is why Copilot Cowork pauses before consequential actions. The tool call is ready to execute — but the system gives you the chance to review before your identity commits to the action.

Responsibility — *be responsible for our actions and deliver on our commitments.*

When Copilot executes a tool call on your behalf, the action happens under your identity. An email sent by Copilot is your email. A document created by Copilot is your document. A record updated by Copilot was updated by you. The delegation of labor does not delegate accountability. Review what Copilot does — because whatever it does, you did.

4. The Model Context Protocol — The Emerging Open Standard

Microsoft Graph and Microsoft's built-in tools are powerful, but they are proprietary to the Microsoft ecosystem. What about connecting to everything else?

The Model Context Protocol (MCP) is the answer — an open standard for connecting AI systems to external tools and data sources, introduced by Anthropic in late 2024 and now adopted across the industry.

4.1 What MCP actually is

MCP standardizes how language models discover, invoke, and receive results from external tools and data sources. Think of it as a universal adapter — a way for any AI system to connect to any tool without custom integration code for every possible combination.

The protocol specifies:

- **Tool discovery.** How a model learns what tools are available and what parameters they accept.
- **Invocation.** How a model requests a tool call with specific arguments.
- **Result handling.** How tool outputs are returned to the model in a format it can incorporate.
- **Context provision.** How external data sources can provide context alongside tool capabilities.

Microsoft has embraced MCP for Copilot extensibility. Copilot Studio supports MCP servers as a way to add custom tools and data sources to agents. This means tools built for MCP work across Copilot, Claude, and other conforming systems — you build once, connect everywhere.

4.2 Why this matters for GES

MCP is the standard that lets you connect GES-specific systems to Copilot without waiting for Microsoft to build a Graph connector for each one.

A custom MCP server for GES could expose:

- **Venue lookup.** Query the venue database for dock configurations, rigging limits, and known constraints.
- **Freight status.** Check the tracking system for shipment location and arrival windows.
- **Labor rate calculation.** Compute labor costs based on jurisdiction, union rules, and shift timing.
- **Exhibitor order retrieval.** Pull the current service order for a given exhibitor and show.

Each of these becomes a tool that Copilot agents can call — structured requests that the MCP server executes against GES systems and returns as grounded context for responses.

The protocol is open source and documented at modelcontextprotocol.io. Building an MCP server requires development work, but the standard means that work produces a reusable asset rather than a point-to-point integration.

5. Copilot Studio — Building Custom Agents with Custom Tools

Understanding the machinery lets you build on top of it. **Copilot Studio** is where that building happens.

5.1 What Copilot Studio provides

Copilot Studio is Microsoft's low-code/no-code platform for creating custom AI agents. These agents can be:

- **Declarative agents for Microsoft 365 Copilot.** Custom agents that appear alongside the built-in Copilot, with their own knowledge and capabilities.
- **Standalone agents.** AI assistants deployed to Teams, websites, or other channels.
- **Agent actions.** Custom tools that extend what agents can do.

The key capability for this chapter is **actions** — the mechanism for adding custom tools to agents.

5.2 Actions — adding tools to agents

An action in Copilot Studio is a tool that an agent can call. Actions can be:

- **Power Automate flows.** Any flow you build becomes a callable tool — query a database, update a record, send a notification, integrate with a third-party API.
 - **Connector actions.** Microsoft's 1,400+ connectors (Salesforce, ServiceNow, SAP, etc.) each expose actions the agent can invoke.
 - **REST API actions.** Upload an OpenAPI specification and Copilot Studio creates actions from your API endpoints.
 - **MCP servers.** Connect to MCP-compliant tool servers for standardized tool access.
-

5.3 A GES worked example — the show operations agent

Here is a concrete example of what this enables at GES.

The scenario: A show operations manager needs quick answers during move-in. They want to ask natural language questions and get actionable responses grounded in real data — without switching between three systems and four browser tabs.

The agent: A custom Copilot Studio agent for show operations, deployed in Teams.

The tools:

1. **Venue knowledge lookup** (Graph connector or MCP server)
 - Input: venue name or code
 - Output: dock configuration, rigging limits, union jurisdiction notes, known constraints
 - Example call: "What are the dock restrictions at the Las Vegas Convention Center?"
2. **Freight status check** (REST API action connected to freight tracking system)
 - Input: exhibitor ID or tracking number

- Output: shipment status, location, ETA, advance warehouse receiving confirmation
 - Example call: "Has Meridian Manufacturing's freight arrived at the advance warehouse?"
3. **Labor rate calculation** (Power Automate flow connecting to labor management system)
- Input: jurisdiction, trade, shift start time, estimated hours
 - Output: calculated cost with applicable rates and rules
 - Example call: "What would overnight carpet labor cost for Hall B?"
4. **Operations manual retrieval** (Graph search scoped to show document library)
- Input: show identifier, topic
 - Output: relevant sections from the operations manual
 - Example call: "What's the move-in sequence for the technology pavilion?"

The interaction:

The show manager opens Teams and types: "Hey Copilot — has the Meridian freight arrived, and what's the dock situation at LVCC for receiving it?"

The agent reasons: this requires two tool calls — freight status for Meridian, and venue knowledge for LVCC. It makes both calls, receives the results, and synthesizes a response: "Meridian Manufacturing's freight arrived at the advance warehouse yesterday at 3:47 PM — 14 crates confirmed. For LVCC, their dock access for this show is Dock 7 through Dock 12, with a 6 AM to 10 PM receiving window. Note: Dock 9 has the freight elevator access, so heavy machinery should route there."

What this changes:

Before: The show manager checks the freight system, then opens the venue specs, then cross-references with the show floor plan. Three systems, five minutes, context switching.

After: One question, one answer, thirty seconds. The manager stays focused on the floor.

Excellence — provide excellent service and execution.

The agent does not replace the show manager's judgment. It retrieves information faster so the manager can apply judgment more often.

Excellence is not the AI answering correctly — it is the manager making better decisions because they have better information access.

6. Governance — Tools Are Attack Surface

Understanding the machinery also means understanding the risks.

6.1 Every tool is a capability and a vulnerability

When you add a tool to an agent, you give it the ability to act. That action happens with real consequences — data is queried, records are updated, messages are sent.

Tool permissions matter. If a Power Automate flow runs with a service account that has elevated access, and that flow is exposed as an agent action, the agent can do things the user asking the question could not do directly. This is a permission escalation risk.

Tool inputs can be manipulated. If a user can phrase a question that causes the agent to call a tool with unexpected parameters, the tool executes those parameters. This is why input validation matters at the tool level, not just the prompt level.

Tool outputs become context. Whatever a tool returns gets incorporated into the agent's response — and potentially into subsequent reasoning. If a tool returns incorrect data, the agent propagates that incorrectness with the same confidence as correct information.

6.2 The governance checklist

For every custom tool or connector deployed at GES:

1. **What permissions does this tool execute with?** If it runs as a service account, does that account have least-privilege access?

2. **What data does this tool return?** Could it return sensitive information to users who should not see it?
3. **What actions can this tool take?** Write operations need more scrutiny than read operations.
4. **What happens if the tool fails?** Does the agent handle errors gracefully, or does it make assumptions?
5. **Who can invoke this tool?** Is it available to all Copilot users or scoped to specific roles?

Every Graph connector brings external data into Copilot's retrieval scope. Every custom action adds a new capability an agent can invoke. Every MCP server exposes new tools.

*This is the point — extending Copilot's reach into GES-specific systems. But extension is also exposure. The governance principle is simple: **treat every new tool like a new API endpoint being exposed to users.** Apply the same security review, the same access control thinking, the same logging and audit requirements.*

A tool that queries exhibitor order data is useful. A tool that queries exhibitor order data and returns it to anyone who asks — including competitors' account teams — is a confidentiality incident waiting to happen.

7. Why Understanding This Makes You Better

This chapter has been technical. Here is why it was worth it.

7.1 You write better prompts

When you understand that Copilot retrieves before it generates, you phrase questions to help retrieval. "What did we discuss in the pre-con meeting about freight targets?" is better than "What are the freight targets?" — because the first question gives retrieval a relationship to follow (the meeting), while the second just gives keywords.

When you understand that the semantic index works by meaning, not keywords, you stop trying to guess exact phrases and start describing what you need in natural language.

7.2 You troubleshoot effectively

When Copilot gives a generic answer, you know why: retrieval found nothing specific, so the model fell back on general knowledge. The fix is not a better prompt — it is better content in the Graph, or scoping your question to a specific source.

When Copilot gives an outdated answer, you know why: stale content in the Graph ranked higher than current content. The fix is archiving the superseded document, not yelling at the AI.

When Copilot says it cannot do something, you know why: it does not have a tool for that action. The fix is building the tool or using a different approach.

7.3 You know what to build

Understanding the architecture shows you where the gaps are. If a show manager's question cannot be answered because the data lives outside the Graph, you know a connector is needed. If a repetitive workflow requires manual execution, you know a Power Automate flow exposed as an action would help.

The difference between "Copilot should do this" and "here is how we would build this" is exactly this architectural understanding.

8. Try This: Trace a Copilot Answer

Pick a Copilot answer you received recently that was actually useful — specific to your work, grounded in real content.

Now trace it backwards:

1. **What did you ask?** Write out your exact prompt.
2. **What content was retrieved?** Look at the citations or sources. Which documents, emails, or conversations did Copilot draw from?

3. **What relationships connected them?** How did those sources relate to your question? Were they authored by relevant people? Shared in relevant channels? Attached to relevant meetings?
4. **What made retrieval succeed?** Was it naming? Was it recency? Was it the relationship structure?
5. **What would have made retrieval fail?** If those documents had been named generically, or filed in a folder nobody uses, or superseded but not archived — would Copilot have found them?

This exercise trains you to see Copilot's answer as an output of the retrieval system, not a product of magic. Once you see it that way, you can influence the inputs.

9. Productive Struggle Problem

You are the GES operations lead for a regional portfolio of trade shows. Your team frequently needs quick answers during move-in that require information from three sources:

- **The venue knowledge base** — dock configurations, rigging limits, union rules (currently in a SharePoint library)
- **The freight tracking system** — shipment status, arrival times, manifest data (currently in a line-of-business application outside Microsoft 365)
- **The labor scheduling system** — crew assignments, rates, availability (currently in another line-of-business application)

Today, answering a question like "What labor do we have scheduled for tomorrow and what freight is still inbound for those exhibitors?" requires three logins and manual cross-referencing.

Your challenge:

1. **Architecture.** Sketch the components needed to make this question answerable through a single Copilot interaction. What stays in SharePoint? What needs a Graph connector? What needs an MCP server or REST API action? What tool calls would the agent need to make?

2. **Data flow.** Trace how the question would flow through the system. What triggers retrieval? What triggers tool calls? How do the results combine into a response?
3. **Permissions.** Who should be able to ask this question? Does the freight tracking system have its own access controls? How do you ensure the Copilot agent respects them?
4. **Failure modes.** What happens if the freight tracking system is down? What happens if the data is stale? What happens if the agent misunderstands and queries the wrong exhibitor?
5. **Governance.** What approval is needed before connecting these systems to Copilot? Who owns the connector or MCP server? How is it audited?

There is no single right answer. The quality of your thinking about *why* each component is needed and *how* they fit together matters more than the specific technology choices.

Glossary

Microsoft Graph

The unified API and data layer underlying all of Microsoft 365.

Organizes organizational data as a graph of nodes (people, documents, meetings, messages, sites) connected by edges (relationships like authored, attended, shared with). The foundation Copilot retrieves from.

Node

In graph terminology, an entity or object. In Microsoft Graph, nodes include people, documents, emails, meetings, calendar events, SharePoint sites, Teams channels, and groups.

Edge

In graph terminology, a relationship between nodes. In Microsoft Graph, edges include "authored," "attended," "shared with," "replied to," "works with," and "belongs to."

Semantic index

The search layer that makes Microsoft Graph content findable by meaning rather than keyword. Converts content into vector embeddings

— mathematical representations of meaning — enabling retrieval of conceptually similar content regardless of exact wording.

Vector embedding

A mathematical representation of content as a point in high-dimensional space. Semantically similar content has vectors that are close together. The semantic index uses vectors to match questions with relevant content.

Retrieval-Augmented Generation (RAG)

The architecture pattern where a language model's response is grounded in retrieved content rather than generated purely from training. Copilot uses RAG — your question triggers retrieval from the Graph, and the results ground the model's answer.

Grounding

The process of providing retrieved content as context for a language model's response. A grounded answer is based on specific sources; an ungrounded answer is based on general training data.

Permission trimming

The enforcement of access controls at retrieval time. When Copilot queries the Graph, it queries as you — only content you have permission to access is returned. Content you cannot see in SharePoint cannot appear in Copilot's answer.

Function calling

The capability of a language model to emit structured requests for external actions rather than just generating text. Also called tool use. The model decides which function to call and with what parameters; a trusted execution layer performs the actual action.

Tool use

Synonym for function calling. The ability of a language model to invoke external tools — APIs, databases, workflows — by emitting structured requests that a separate system executes.

Tool call

A single invocation of a tool by a language model. Consists of the tool name and structured parameters. The model does not execute the tool; it requests execution.

Model Context Protocol (MCP)

An open standard introduced by Anthropic for connecting AI systems to external tools and data sources. Standardizes tool discovery, invocation, and result handling so tools built for MCP work across multiple AI platforms.

Graph connector

A component that ingests data from external systems into Microsoft Graph and the semantic index. Makes line-of-business data retrievable alongside Microsoft 365 content. Available as pre-built connectors for common systems or buildable custom via the connector SDK.

Copilot Studio

Microsoft's platform for building custom AI agents. Provides no-code/low-code tools for creating agents with custom knowledge, custom actions (tools), and deployment to Teams, websites, or Microsoft 365 Copilot.

Action (Copilot Studio)

A tool that a Copilot Studio agent can invoke. Can be a Power Automate flow, a connector action, a REST API call, or an MCP server connection.

Declarative agent

A custom agent built in Copilot Studio that extends Microsoft 365 Copilot with specialized knowledge and capabilities while appearing alongside the built-in Copilot experience.

Context window

The maximum amount of content (measured in tokens) that a language model can process in a single interaction. Retrieved content, conversation history, and the current prompt all compete for space in the context window.

Grounding drift

The degradation of answer quality caused by stale content in the Graph. Superseded documents that remain indexed can be retrieved and presented as current, producing confidently incorrect answers.

Discussion

Understanding how Copilot actually works changes the relationship. You stop being a user hoping for good results and start being an operator who can diagnose problems and influence outcomes.

Consider your own work at GES. Where are the gaps between what Copilot can answer today and what would be genuinely useful? Are those gaps in the Graph (missing content or poor organization), in tools (actions that don't exist yet), or in understanding (people not knowing how to ask)?

Post your reflection in the course discussion forum. Your response should:

- *Identify one question you frequently need answered during show operations that Copilot currently cannot answer well. Diagnose why — is the data outside the Graph? Is there no tool for the required action? Is the content in the Graph but poorly organized or stale?*
- *Propose a solution using the concepts from this chapter. Would a Graph connector help? A custom Copilot Studio action? Better SharePoint governance? Be specific about which component would address which gap.*
- *Address the governance implications of your proposal. What permissions would be needed? Who would own the connector or action? What could go wrong?*
- *Respond to at least **two peers** with substantive engagement — challenge an architectural decision, point out a governance gap, or suggest an alternative approach.*

Minimum 300 words for your main post.

Leader's Takeaway

This chapter gave your team the technical foundation to become genuinely good at AI — not just users, but operators who understand the machinery.

Three strategic implications for GES leadership:

First: The Graph is the asset. Copilot's value is directly proportional to the quality of data in Microsoft Graph. Every investment in SharePoint governance, document organization, and content currency is an investment in AI capability. Conversely, every permission gap or stale document degrades AI quality across the organization. The Graph is not IT infrastructure — it is organizational intelligence infrastructure.

Second: Graph connectors are the competitive lever. Microsoft 365 data is table stakes — every company using Copilot has it. GES's advantage lies in connecting the systems competitors don't have: venue knowledge accumulated over decades, labor scheduling refined through thousands of shows, freight operations patterns learned the hard way. Connectors that bring this data into the Graph make Copilot uniquely valuable at GES. This is where IT investment has outsized ROI.

Third: Tool governance is the discipline that makes tools safe. Every custom action, every Graph connector, every MCP server is a new capability — and a new attack surface. The instinct to move fast on tools must be balanced by the discipline to govern them properly. Permissions, access controls, audit logging, and ownership accountability apply to AI tools exactly as they apply to any API endpoint. The people building tools need to be talking to the people who own security and compliance.

The chapter's final message is this: understanding how Copilot works is not technical curiosity. It is operational competence. The leaders who build teams that understand the Graph, retrieval, and tools will outperform the leaders who treat AI as a black box to prompt harder. Understanding is leverage.

Resources for Chapter 17

- *Microsoft Graph Documentation:* learn.microsoft.com/graph
(<https://learn.microsoft.com/en-us/graph/>)
- *Semantic Index for Microsoft 365 Copilot:*
learn.microsoft.com/microsoftsearch/semantic-index-for-copilot
(<https://learn.microsoft.com/en-us/microsoftsearch/semantic-index-for-copilot>)
- *Graph Connectors Overview:*
learn.microsoft.com/graph/connecting-external-content-connectors-overview
(<https://learn.microsoft.com/en-us/graph/connecting-external-content-connectors-overview>)
- *Copilot Studio Documentation:* learn.microsoft.com/microsoft-copilot-studio
(<https://learn.microsoft.com/en-us/microsoft-copilot-studio/>)
- *Model Context Protocol:* modelcontextprotocol.io
(<https://modelcontextprotocol.io/>)
- *Function Calling Overview (Martin Fowler):*
martinfowler.com/articles/function-call-LLM.html
(<https://martinfowler.com/articles/function-call-LLM.html>)
- *GES:* ges.com (<https://ges.com>)